

Week 3: RNN, LSTM, and GRU

Week 3 asks a new architectural question: **how do we model data whose meaning depends on order over time?**

- Focus: sequence order, hidden state, recurrence, long-term dependency, gated memory
- Output: students should be able to explain why RNNs were introduced and why they still hit bottlenecks
- Bridge sentence: sequence models add memory over time, not just more static nonlinear layers

Where Week 3 Fits

Week 3 adds memory over ordered inputs on top of Week 1 representation and Week 2 learning dynamics.

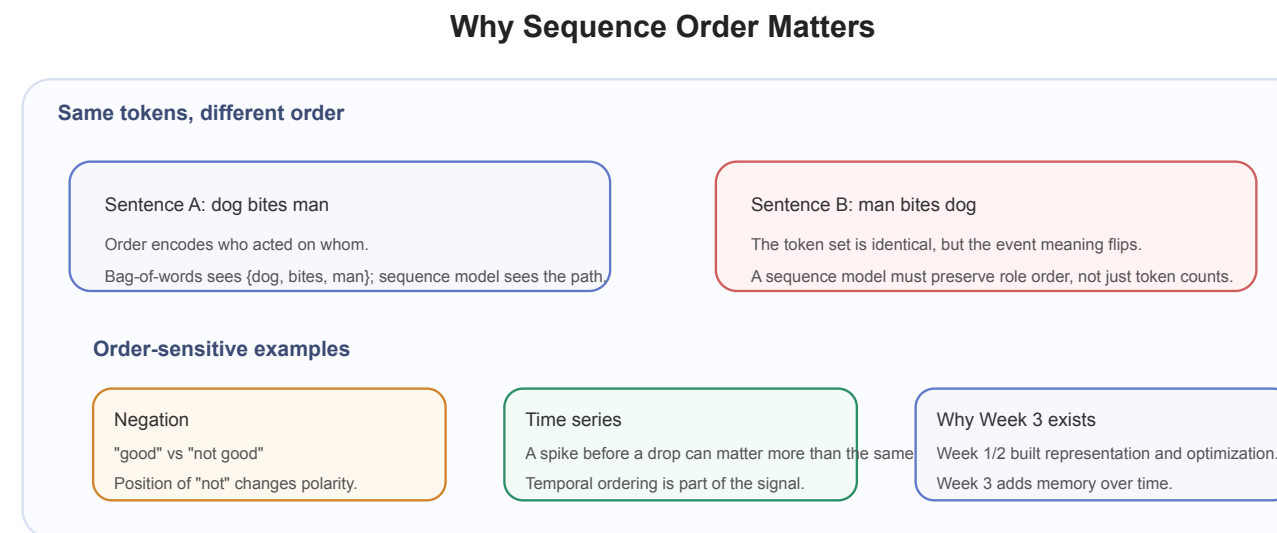
Roadmap

Week	Core theme	Student takeaway
1	MLP, activation, loss	Why nonlinear representation matters
2	Gradient, backprop, optimizer	How loss becomes parameter updates
3	RNN/LSTM/GRU	How hidden state carries sequence information
4	Attention	How models revisit tokens directly
5	Transformer + paper reading	How to read architecture and experiments

Why Order Matters

A strong bag-of-words baseline can ignore order, but many tasks change meaning when token positions change.

Order-sensitive examples



- **dog bites man** and **man bites dog** contain the same tokens but imply different events
- Negation and temporal ordering often matter more than raw token counts
- Week 3 starts where classical feature counts stop being enough

Why Baselines Still Matter

Sequence models should beat or complement a baseline for a reason, not by assumption.

Baseline comparison

Approach	Strength	Limitation on ordered data
TF-IDF + Logistic Regression	fast, interpretable, strong low-data baseline	token order is mostly lost
MLP on pooled features	nonlinear classifier	still weak at explicit time dependence
RNN family	processes input step by step	training and memory bottlenecks remain

RNN Cell in One Update Rule

An RNN reuses the same recurrent cell across time, combining the new input with the previous hidden state.

Recurrent update

$$\begin{aligned}h_t &= \phi(W_x x_t + W_h h_{t-1} + b) \\ y_t &= W_y h_t + c\end{aligned}$$

- x_t : current input token or frame at time step t
- h_t : task-specific summary of what the model wants to carry forward
- The same parameters are reused at every time step

Hidden State and Tensor Shapes

Sequence models only become easy to debug when students can track **batch**, **time**, and **feature** dimensions explicitly.

Shape discipline

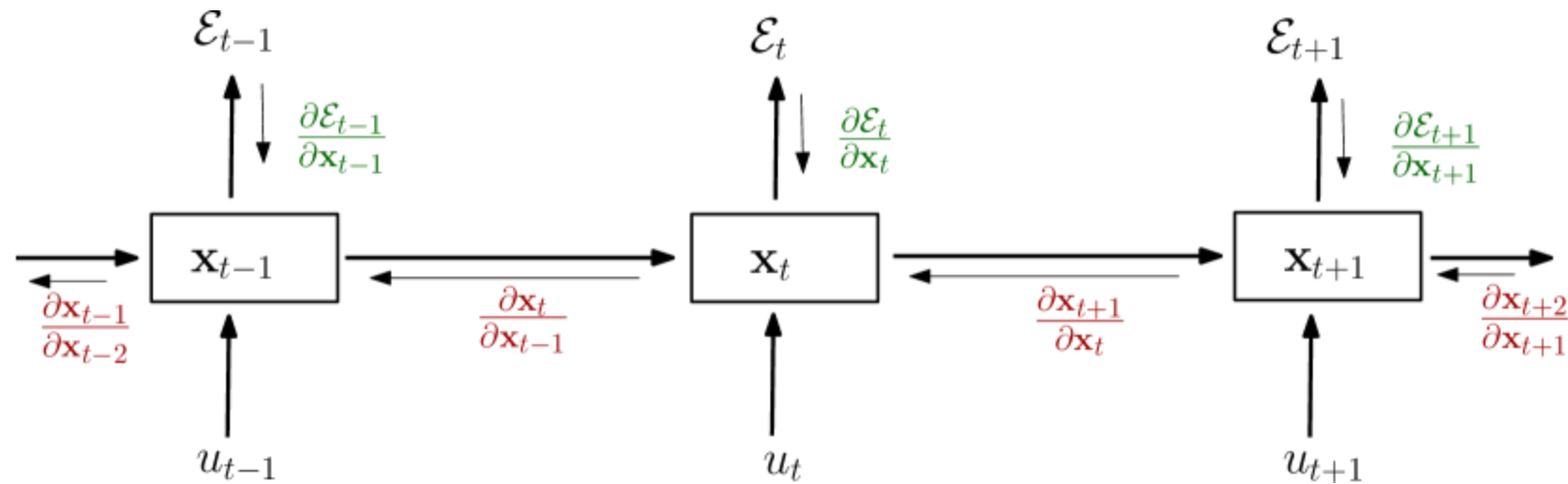
```
Missing \end{aligned}  
\ H &:  
batch, time,  $d_{hidden}$   
\ h_t &:  
batch,  $d_{hidden}$   
\end{aligned}]
```

- The last hidden state is often used for many-to-one classification
- The full hidden sequence can be used for tagging or later attention
- Week 3 is as much about shape literacy as about new layers

Unrolled RNN Over Time

A recurrent model is easier to understand when unfolded into a long computational graph over time steps.

Pascanu et al. 2013, Fig. 2: unrolling an RNN



- This is the actual paper figure used to motivate BPTT in [On the difficulty of training Recurrent Neural Networks](#)
- One recurrent module becomes many time-indexed copies in the training graph
- The error signal must travel through this chain, which sets up vanishing/exploding gradients

Many-to-One vs Many-to-Many

Sequence models can produce one output at the end or a structured output across many time steps.

Sequence task patterns

Pattern	Example	What is read out?
many-to-one	sentiment or video classification	last hidden state or pooled representation
many-to-many	tagging or frame-wise video labels	hidden state at each time step
encoder-decoder preview	translation	compressed source state or later attention context

Why Long-Term Dependency Is Hard

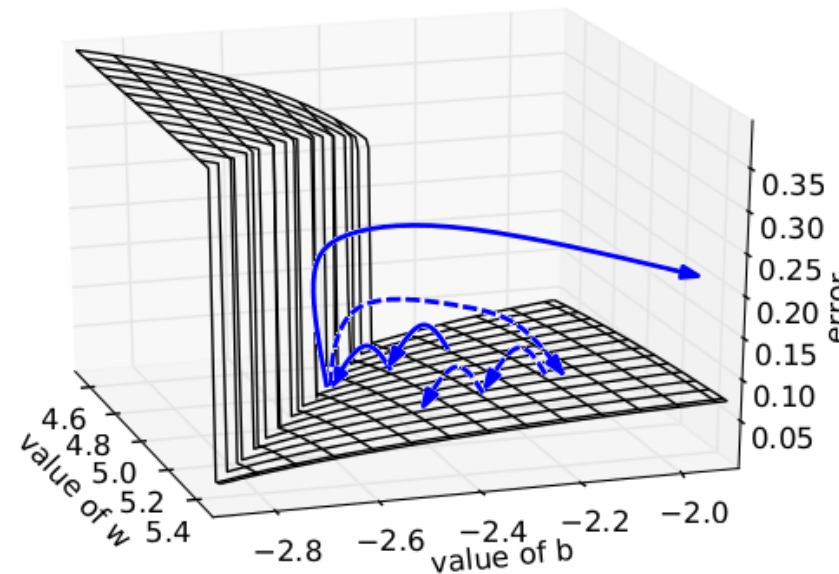
If an early token matters much later, the model must preserve its influence through many recurrent updates.

- The hidden state is not a perfect memory buffer; it is a learned compression of the past
- Important evidence can be overwritten or diluted as new inputs arrive
- This is the hidden-state bottleneck that motivates gated memory and later attention

Vanishing/Exploding Gradient

Long time horizons make the gradient travel through repeated Jacobian multiplications, so the signal can shrink or blow up.

Pascanu et al. 2013, Fig. 6: exploding gradients and clipping



- The figure makes exploding gradients concrete: updates can jump across steep curvature walls
- Gradient clipping keeps the direction but limits the norm when it becomes too large
- Vanishing gradients are the complementary problem: the signal becomes too small to reach early time steps

RNN Bottleneck Summary

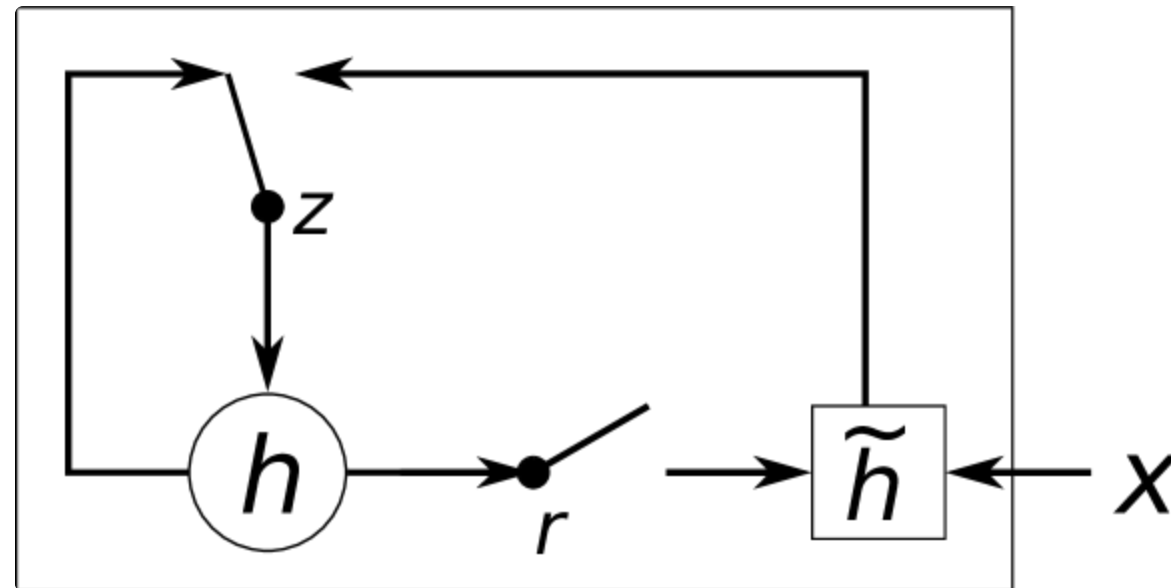
RNNs solve the order problem better than bag-of-words, but they pay for it with sequential computation and memory compression.

- Information flows through a single hidden-state path
- Long dependencies require many recurrent hops
- Training is harder because gradients propagate across a long time graph
- 2025-2 CS231n bridge: CNNs see local temporal windows, while RNN states summarize all previous vectors
- Sequential recurrence is hard to parallelize, motivating LSTM/GRU first and attention next

Why Gated Memory Was Introduced

LSTM and GRU make memory flow more selective: they learn what to keep, what to update, and what to expose.

Cho et al. 2014, Fig. 2: gated hidden unit later known as GRU

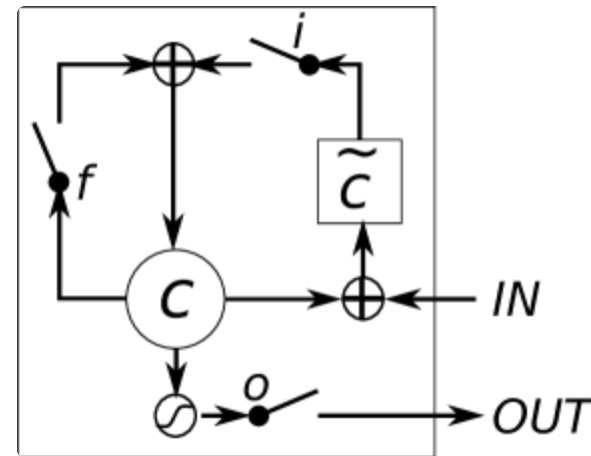


- Update gate z : how much previous hidden state should carry forward?
- Reset gate r : when should the unit ignore old state and recompute from the current input?
- The figure turns **memory** into a controlled flow diagram rather than a metaphor

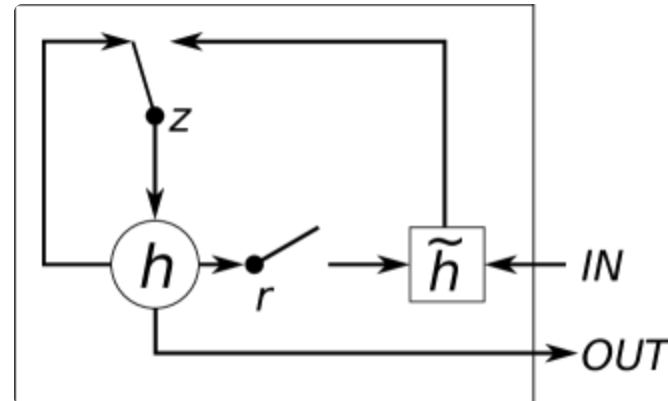
RNN, LSTM, and GRU at a Glance

The main difference across the family is how explicitly memory is controlled and protected.

Chung et al. 2014, Fig. 1a: LSTM unit



Chung et al. 2014, Fig. 1b: GRU unit



Comparison Table

No recurrent variant is universally best; each one trades simplicity, control, and cost differently.

RNN family trade-offs

Model	Strength	Weakness	Teaching takeaway
RNN	simple and conceptually clean	weak on long dependencies	best for introducing recurrence
LSTM	stronger memory control	more parameters and complexity	good default when long context matters
GRU	lighter gating structure	not always more accurate than LSTM	practical compromise

Experiment Design for Week 3

The right experiment compares sequence-aware models against a strong non-sequence baseline on the same task.

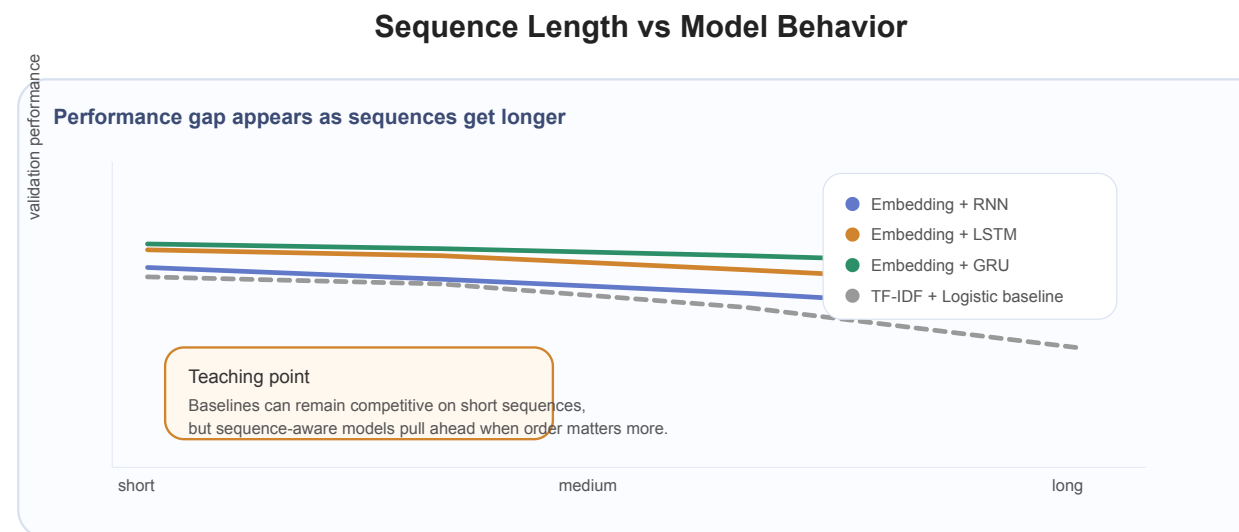
Suggested comparison

Model	Input view	What we want to learn
TF-IDF + Logistic Regression	unordered feature counts	baseline strength without order
Embedding + RNN	ordered tokens with simple recurrence	what recurrence alone adds
Embedding + LSTM	ordered tokens with gated memory	whether longer memory helps

Sequence Length Effect

Sequence-aware models often earn their keep on longer or more order-sensitive inputs, not always on short easy cases.

Length-sensitive comparison



- Short sequences may leave a strong baseline surprisingly competitive
- The gap often grows when long-range context matters more
- This sets up Week 4: what if the model could revisit tokens directly?

PyTorch Sequence Classifier

Students should map the high-level idea to actual layers, hidden states, and tensor outputs.

Minimal LSTM classifier

```
embed = nn.Embedding(vocab_size, d_model)
encoder = nn.LSTM(input_size=d_model, hidden_size=hidden_dim, batch_first=True)
classifier = nn.Linear(hidden_dim, num_classes)

emb = embed(x)
out, (h_n, c_n) = encoder(emb)
logits = classifier(h_n[-1])
```

- **batch_first=True** keeps the shape convention readable
- **h_n[-1]** is a common summary for many-to-one classification
- 2025-2 LSTM notebooks: MNIST rows can be treated as **sequence_length = 28**; stock/power data use sliding windows
- Logging sequence length buckets can expose failure patterns hidden by one average metric

Checkpoint to Attention

By the end of Week 3, students should be able to explain why sequence order created a new model family and why that family still hit a bottleneck.

- RNNs process sequence order by updating a hidden state over time.
- Long-term dependency is hard because memory and gradient signals must survive many recurrent steps.
- LSTM/GRU introduce gates to manage memory more carefully.
- RNNs are sequential, so long sequence processing is difficult to parallelize.
- The next question is whether the model can revisit all tokens directly instead of compressing everything into one state.

